

Coursework 1 — Yahtzee

1. Overview

The aim of this coursework is to implement a version of dice game [Yahtzee](#) in C#. The description and rules of the game you will be implementing can be found in Appendix B.

This is an **individual project**, and you will have to submit your own, original solution for this coursework specification, consisting of a report, the source code, an executable, and a video report showcasing all implemented features.

The **learning objective** of this coursework is for students to develop proficiency in advanced programming concepts, and to apply these programming skills to a concrete application of moderate size. Design choices regarding tools and libraries chosen for the implementation need to be justified in the accompanying report.

This coursework will develop personal abilities in autonomous problem analysis, the usage of a modern object-oriented language for system programming, and deepen the understanding of integrating components in the .NET framework (primarily targeting Windows systems, but supported on other OS-es too).

2. Lab Environment

For this coursework you should use Visual Studio C# 2026, which will be the reference platform for testing. You can obtain a full version through [Azure for Education](#) using your Heriot-Watt student credentials. Alternatively, you can use [VS Code](#) IDE for developing your application. VS Code supports development under GNU/Linux and MacOS systems, as well as Windows.

For the GUI component, use a library of your choice. Under Visual Studio, the [Windows Forms](#) libraries are recommended as the simplest starting point. The main downside of Windows Forms is that they are an older UI library, available exclusively for Windows systems. Alternatively, consider using [MAUI](#), a modern cross-platform UI library. Of course, these recommendations are just recommendations. Treat them as a starting point in your exploration of which library to use.

If you develop the software in a different environment, you must make sure that the code is compatible with the above software. Beware of incompatibilities due to different versions of the IDE or compiler that you use. Make sure that you can run the final application on a Windows lab machine.

For technical HOWTOs about accessing software of relevance for this course, see the [technical HOWTOs](#) on the course's Canvas page.

The submission of the final software must be in form of both *complete sources* as well as a *standalone executable* that can be installed and executed on any Windows 11 (with .NET Core) system. If external tools are used, they need to be included in this executable. Any dependencies on tools or external software must be clearly marked in the report.

3. Functional Requirements:

The goal of the game is to score points by rolling five dice to make certain combinations. The dice can be rolled up to three times in a round to try to make various scoring combinations. The game lasts for 13 rounds, during which players fill out 13 different scoring categories.

The game can be played as a single-player or a multi-player game. When played as a multi-player game, each player has their own scoring sheet, and players take turns rolling the dice (i.e., once a player finishes their rolls and chooses which category to score, the play proceeds to the next player).

Detailed game rules are available in Appendix B.

The rest of the section specifies the desired features using the [MoSCoW prioritisation framework](#).

3.1. Basic Gameplay

- The application *must* provide a **graphical user interface (GUI)** for starting, playing, saving, and loading games.
- The program *must* enable **single-player** games to be played.
- When starting a new game, the program *should* enable hotseat¹ **multi-player** games. You are allowed to limit the maximal number of players in a multi-player game.
- The program *must* enable the user to specify the name of each player.
- During the game, the GUI *must* make it clear **which player's turn** is it to roll the dice and **how many rolls they have available**.
- On player's second and third roll, the GUI *must* make it clear **which dice will be rolled** and which ones are set aside.
- After each roll, the program *should* provide a **score preview** for each available scoring category.
- It *should* be possible to **save the game** after any roll made by the player. For this feature, design a save file format that will allow you to resume the game from the saved position.
- If attempting to exit the program during a game in progress without first saving the game, the program *should offer the user the option to first save the game before exiting*.
- At the end of the game, the program *must* display a **summary of results**, consisting of player's names and scores, ordered by the achieved score.

3.2. Advanced Features

- **Games Database**
 - The program *should* maintain the database of all completed games, recording when was the game played, players' names, and their scores.
 - The games database *should* be kept using a database engine.
 - The program *should* enable the user to access the information about five highest overall scores and five highest overall scores of a particular player.
 - When reporting the high-scores, the program *could* include the date on which each of the high-scores was achieved.
- **AI opponents**
 - The program *should* enable playing against computer-controlled opponents. (For this feature, even the most trivial playing strategy is ok.)
 - The program *could* have computer-controlled opponents with different gameplay strategies. (If you have time, have fun coming up with good AI strategies.)

4. Report Format

The report should be up to 10 pages long and use the following format (if you need space for additional screenshots, put them into an appendix, not counting against the page limit, but don't rely on the screenshots in your discussion):

1. **Introduction:** state the purpose of the report, your remit, and any assumptions you have made during the development process.

¹Multiplayer mode in which the players take turns playing on a single instance of the game.

2. **Requirements checklist:** clearly state which requirements are implemented and which are not; this should be in the form of a table (as shown in Appendix A) listing all the required features, and stating whether the functionality is fully implemented, partially implemented, or missing.
3. **Design Considerations:** describe the basic design decisions you have made in developing your code, covering class design, choice of data structures, GUI design, other performance-relevant choices etc. In particular, make sure to **design clear class and method interfaces** and document them in the report (see the next point).
4. **Developer guide:** describe your application design and main areas of code in order to help another developer understand your work and how they might continue the development. In particular, point out
 - how is your code organized in classes, and which class is responsible for which functionality;
 - how the class and method interfaces encapsulate different functionalities;
 - specification of the save-file format (if implemented); and
 - how is the game database organized (if implemented).
5. **Generative AI usage statement:** if you used generative AI while working on this coursework (see Section 7 for GenAI policy on this course), describe how was it included in your workflow. In particular, include any prompts and model's responses as appendices. (Appendices do not count towards the report's page limit.)
6. **Conclusion:** reflect on what you are most proud of in the application and what you would have liked to have done differently.
7. An optional final section of references is also encouraged. The list of references does not count toward the report's page limit.

5. Video Demonstration Format

The video demonstration must showcase all the implemented and partially implemented features of the application, matching the requirement checklist provided in the report. **Features not demonstrated in the video will be considered not implemented.**

Do not concern yourself with making high-quality production video. All that is important is that the functionality of the application is clearly visible.

The video showcasing the use of the application is **mandatory** (barring any mitigating circumstances). **Without the video the submission is incomplete** and may be marked as 0 points.

6. Submission and Marking Scheme

The coursework submission consists of

1. A zip archive containing
 - the source code;
 - a stand-alone executable; and
 - the report (in pdf format).
2. A video of running the application, with an explanation as voice-over audio. Either upload the video on Canvas or provide a OneDrive link to the video in the report.
3. If deemed necessary, you might be asked to give an in-person demonstration of your submission.

The standard penalty of 30% of the maximum available mark applies to late submissions. No submissions will be accepted after 5 working days beyond the submission deadline. Submission must be through Canvas. **This coursework is worth 50%** of the final mark. The marking scheme for this project can be seen below.

Marking Scheme	
Criterion	Marks
Single-player gameplay	5
Multi-player gameplay	10
Score preview	5
Saving/loading game	10
GUI quality	5
Games Database	15
AI opponents	10
Code organisation and quality (portable coding, clear method interfaces, sufficient comments, sensible variable/object names, good code/class/method design, use of advanced language features)	20
Report quality (Content and communication, design considerations, reflections, developer guide, feature checklist)	15
Video demonstration of running the application with explanation as voice-over audio	5
Total marks	100

7. Generative AI Usage Policy

Our policy on the use of generative AI tools in F20SC is as follows:

You may use GenAI to complete specified tasks you then evaluate this output yourself with no GenAI assistance to provide the assessment submission. In particular, you are allowed to utilize Copilot and IntelliCode features included in Visual Studio 2026, as these tools are commonly relied on in industrial practice.

This policy corresponds to [Level 6 of the University GenAI usage levels](#).

Please note that you are responsible for the quality of the code you submit, and you have to be able to provide a detailed developer's guide as well as **be able to discuss and explain the submitted code**, regardless of how much you relied on generative AI.

8. Academic Integrity

Your attention is drawn to the university policy on academic misconduct and to the Contract Cheating Risk and Avoidance Guide (links available via Canvas).

This is a **individual project** and you will have to submit your own, original solution for this coursework specification, consisting of a report, the source code and an executable.

The code you submit must be the your own, or if some text or code in the coursework has been taken from other sources (beyond the material provided for this course), you must give a clear reference to the source in comments in the code or the report.

Failure to clearly reference work that has been obtained from other sources, or copying the words or code of another student, is plagiarism.

Asking a third party to complete coursework on your behalf, and then submitting it as your own work, is contract cheating.

You must never give a copy of your coursework writing or code to another student, and you must always refuse any request from another student for a copy of your coursework. Sharing your coursework with another student is collusion.

Suspected plagiarism, contract cheating or collusion will be referred to the School Discipline Committee for investigation. This will delay your feedback, and if you are found guilty of misconduct then it will result in a penalty such as voiding the course or being withdrawn from the university.

Please don't do any of the things above! If you have any concerns about potential academic misconduct, please ask your local course leader for guidance.

For details see [University's academic misconduct policy](#) and [Student Guide to Academic Integrity](#).

A. Feature Checklist Example

Here is an example feature checklist, listing all the features that will be marked and their implementation status. Partially implemented features come with a small note describing the known issues.

FEATURE	STATUS
Single-player gameplay	fully implemented
Multi-player gameplay	fully implemented
Specifying player names	fully implemented
Displaying whose turn is it	fully implemented
Displaying the number of rolls available	not implemented
Indicating which dice will be rolled/set aside	fully implemented
Score preview	not implemented
Saving/loading games	partially implemented – Works only for single-player games
Offer to save the game before exiting	partially implemented – Always offers to save, regardless of whether the current state has already been saved.
Result summary at the end of the game	fully implemented
Games database	partially implemented – Games saved in SQLite database, but the support for reading information from the database is missing
AI opponents	not implemented

The feature checklist in your report must list exactly the features listed in the above example. Of course, the status column should reflect the state of your submission.

B. Simplified Yahtzee Game Rules

Here is a simplified set of Yahtzee rules, adapted from <https://en.wikipedia.org/wiki/Yahtzee>. **Your task is to implement these simplified rules**, not the full rules of Yahtzee.

The game consists of 13 rounds. In each round, a player gets three rolls of the dice, although they can choose to end their turn after one or two rolls. The player begins the round by rolling five six-sided dice. After the first and second rolls the player can save any dice they want and re-roll the rest. The player has complete choice as to which dice to roll. It is possible to re-roll any dice that were or were not rolled before. After a player ends their round, the play passes to the next player.

The Yahtzee scorecard contains 13 different category boxes, as shown in Fig. 1. After completing each round, the player must enter a score in one of these category boxes. The score entered in the box depends on how well the five dice match the scoring rule for the category. If a category is chosen but the dice do not match the requirements of the category the player scores 0 in that category. Once a score has been entered for a category, that category may not be used again by that player in that game. The game is completed after 13 rounds by each player, with each of the 13 boxes filled. Total scores are calculated by summing all thirteen boxes. The player with the highest score wins.

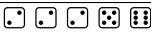
Category	Description	Score	Example
Aces	Any combination	The sum of dice with the number 1	 scores 3
Twos	Any combination	The sum of dice with the number 2	 scores 6
Threes	Any combination	The sum of dice with the number 3	 scores 12
Fours	Any combination	The sum of dice with the number 4	 scores 8
Fives	Any combination	The sum of dice with the number 5	 scores 5
Sixes	Any combination	The sum of dice with the number 6	 scores 18
Chance	Any combination	Sum of all dice	 scores 14
Three of a Kind	At least three dice the same	Sum of all dice	 scores 17
Four of a Kind	At least four dice the same	Sum of all dice	 scores 24
Full House	Three of one number and two of another	25	 scores 25
Small Straight	Four sequential dice	30	 scores 30
Large Straight	Five sequential dice	40	 scores 40
Yahtzee	All five dice the same	50	 scores 50

Figure 1: Yahtzee scorecard. Note that a valid yahtzee is *not* a valid full house.